



ÉCOLE NORMALE SUPÉRIEURE DE L'ENSEIGNEMENT
TECHNIQUE DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA

ÉCOLE NORMALE SUPÉRIEURE DE L'ENSEIGNEMENT TECHNIQUE - MOHAMMEDIA

ALGORITHMIC COMPLEXITY & BENCHMARKING

Comparative Analysis of Sorting Algorithms

Realised by :

Mohamed Ed Deryouch
Mousaab El Harmali

Supervised by :

Prof. QBADOU

December 13, 2025

Contents

1	Introduction	2
2	Theoretical Background	2
2.1	Big O Notation	2
2.2	Stability	2
2.3	Memory Usage (In-place vs. Out-of-place)	3
3	Algorithm Analysis	3
3.1	Quadratic Algorithms ($O(n^2)$)	3
3.1.1	Bubble Sort	3
3.1.2	Selection Sort	4
3.1.3	Insertion Sort	4
3.2	Efficient Algorithms ($O(n \log n)$)	4
3.2.1	Merge Sort	5
3.2.2	Heap Sort	5
3.3	Quick Sort Variants	5
3.3.1	Quick Sort V1 (Standard)	5
3.3.2	Quick Sort V2 (Optimized)	5
3.4	Tim Sort	5
4	Experimental Setup	6
4.1	Environment	6
4.2	Dataset	6
5	Performance Results and Analysis	6
5.1	Quantitative Results	6
5.2	Visual Analysis	7
5.3	Discussion	8
5.3.1	The $O(n^2)$ Wall	8
5.3.2	Quick Sort Optimization (V1 vs V2)	8
5.3.3	Tim Sort vs. The Rest	8
5.3.4	Key Insights at 100,000 Elements	9
6	Conclusion	9

1 Introduction

Sorting is one of the most fundamental and widely studied operations in computer science. It serves as a critical building block for numerous applications, including database management, search algorithms, and data compression. The choice of a sorting algorithm can have a profound impact on the overall performance of a system, especially when dealing with large datasets.

The primary objective of this project is to implement, verify, and benchmark a variety of sorting algorithms to understand their theoretical and practical performance characteristics. By comparing algorithms from different complexity classes ($O(n^2)$ vs. $O(n \log n)$), we aim to provide a concrete demonstration of how algorithmic efficiency scales with input size.

This report details the implementation of eight distinct algorithms, the methodology used for benchmarking, and a rigorous analysis of the results. We utilize a custom C benchmarking suite that ensures precise timing and correctness verification.

The complete source code for this project, including the benchmarking suite and algorithm implementations, is available on GitHub: <https://github.com/simoderyouch/Sorting-Algorithms-in-C>.

2 Theoretical Background

Before analyzing the specific algorithms, it is essential to establish the theoretical framework used for comparison.

2.1 Big O Notation

Big O notation is used to classify algorithms according to how their run time or space requirements grow as the input size grows.

- $O(n^2)$ (**Quadratic**): The execution time grows proportionally to the square of the input size. Doubling the input quadruples the time. Common in simple algorithms like Bubble Sort.
- $O(n \log n)$ (**Linearithmic**): The execution time grows in proportion to n multiplied by the logarithm of n . This is significantly faster than quadratic growth for large n and is the standard for efficient sorting.
- $O(n)$ (**Linear**): The execution time grows linearly with input size. Best case for some adaptive algorithms.

2.2 Stability

A sorting algorithm is considered **stable** if it preserves the relative order of equal elements. If two items compare as equal, the one that appeared first in the original input will appear first in the sorted output. This property is crucial when sorting by multiple keys (e.g., sorting a list of people by last name, then by first name).

2.3 Memory Usage (In-place vs. Out-of-place)

- **In-place:** The algorithm sorts the data within the original array, requiring only a small, constant amount of extra auxiliary memory ($O(1)$ or $O(\log n)$ stack space).
- **Out-of-place:** The algorithm requires extra memory proportional to the input size ($O(n)$) to hold temporary data, as seen in standard Merge Sort.

3 Algorithm Analysis

3.1 Quadratic Algorithms ($O(n^2)$)

These algorithms are generally easy to implement but suffer from poor performance on large datasets.

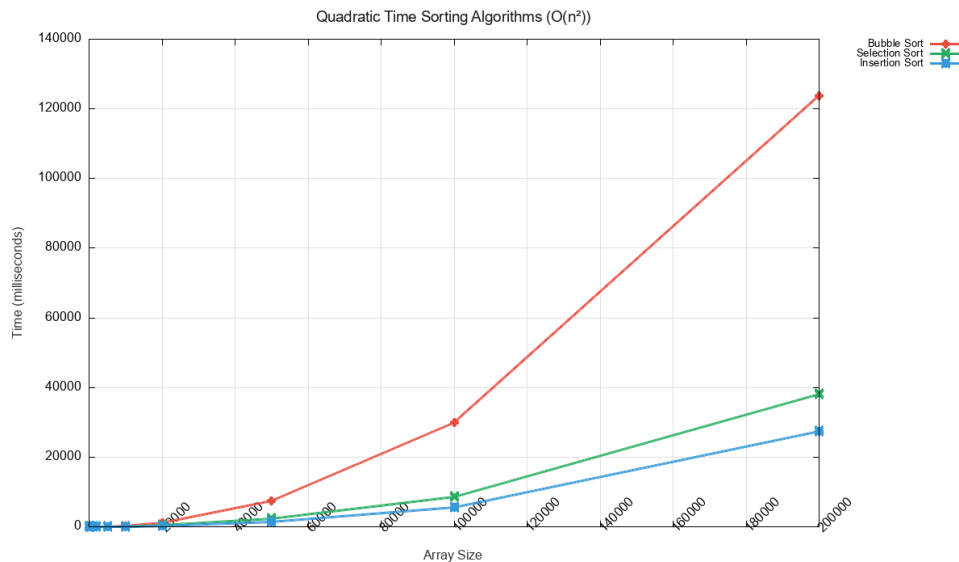


Figure 1: Performance of Quadratic Algorithms ($O(n^2)$).

The execution time increases drastically as the input size grows, illustrating the exponential nature of these algorithms. Bubble Sort is particularly inefficient for large datasets.

3.1.1 Bubble Sort

Mechanism: Repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted.

- **Complexity:** Best $O(n)$, Average/Worst $O(n^2)$.
- **Pros:** Simple, stable, detects sorted lists.
- **Cons:** Extremely slow for large unsorted data.

3.1.2 Selection Sort

Mechanism: Divides the input into a sorted and unsorted region. It repeatedly finds the minimum element from the unsorted region and moves it to the end of the sorted region.

- **Complexity:** $O(n^2)$ in all cases.
- **Pros:** Simple, minimizes swaps (at most n swaps).
- **Cons:** Unstable, slow.

3.1.3 Insertion Sort

Mechanism: Builds the sorted array one item at a time. It takes an element from the unsorted portion and inserts it into its correct position in the sorted portion.

- **Complexity:** Best $O(n)$, Average/Worst $O(n^2)$.
- **Pros:** Stable, efficient for small or nearly sorted datasets, low overhead.
- **Cons:** Slow for large random datasets.

3.2 Efficient Algorithms ($O(n \log n)$)

These algorithms utilize the divide-and-conquer paradigm to achieve superior scalability.

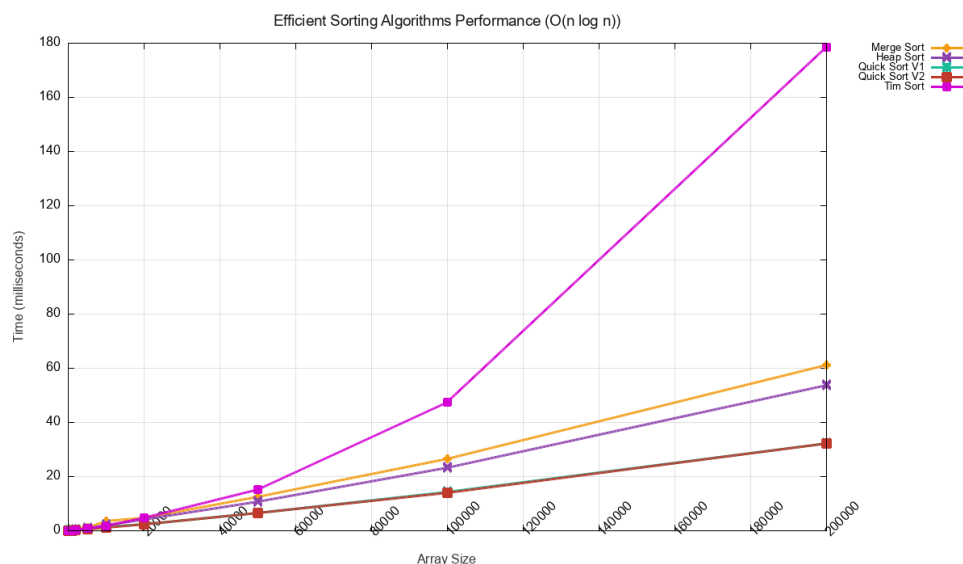


Figure 2: Performance of Efficient Algorithms ($O(n \log n)$).

These algorithms demonstrate much better scalability. The execution time grows almost linearly, making them suitable for large datasets.

3.2.1 Merge Sort

Mechanism: Recursively divides the array into halves until single elements remain, then merges the sub-arrays back together in sorted order.

- **Complexity:** $O(n \log n)$ in all cases.
- **Pros:** Stable, consistent performance, parallelizable.
- **Cons:** Requires $O(n)$ extra space.

3.2.2 Heap Sort

Mechanism: Converts the array into a binary heap structure. It repeatedly extracts the maximum element from the heap and places it at the end of the array.

- **Complexity:** $O(n \log n)$ in all cases.
- **Pros:** In-place ($O(1)$ extra space), consistent performance.
- **Cons:** Unstable, typically slower than Quick Sort due to poor cache locality.

3.3 Quick Sort Variants

Quick Sort selects a 'pivot' and partitions the array into elements less than and greater than the pivot.

3.3.1 Quick Sort V1 (Standard)

Mechanism: Standard recursive implementation.

- **Issue:** In the worst case (sorted array with bad pivot choice), recursion depth can reach $O(n)$, potentially causing a stack overflow.

3.3.2 Quick Sort V2 (Optimized)

Mechanism: Implements **Tail Recursion Elimination**. After partitioning, the algorithm recursively calls itself for the smaller partition and loops back to handle the larger partition.

- **Benefit:** Guarantees a stack depth of $O(\log n)$, preventing stack overflow and reducing function call overhead. This makes it robust for production use.

3.4 Tim Sort

Mechanism: A hybrid algorithm derived from Merge Sort and Insertion Sort. It divides the array into small chunks called "runs", sorts them with Insertion Sort, and then merges them using a smart merging strategy.

- **Complexity:** Best $O(n)$, Average/Worst $O(n \log n)$.
- **Pros:** Stable, extremely fast on real-world data (which often has ordered subsequences), standard in Python and Java.

4 Experimental Setup

4.1 Environment

- **Language:** C (C99 Standard)
- **Compiler:** GCC
- **Timing:** `clock()` function from `time.h` (measures CPU time).

4.2 Dataset

We utilized random integer arrays to simulate a generic unsorted dataset.

- **Sizes:** 100, 500, 1000, 2000, 5000, 10000, 20000, 50000, 100000, 200000.
- **Range:** Values between 0 and $size \times 10$.
- **Verification:** A verification step runs after each sort to ensure the array is strictly increasing.

5 Performance Results and Analysis

5.1 Quantitative Results

Table 1 presents the raw execution times.

Size	Bubble	Selection	Insertion	Shell	Merge	Heap	Quick V1	Quick V2	Tim
100	0.025	0.013	0.007	0.007	0.024	0.011	0.007	0.007	0.010
500	0.719	0.313	0.200	0.058	0.182	0.068	0.062	0.066	0.076
1000	3.329	1.285	0.727	0.129	0.364	0.171	0.111	0.115	0.141
2000	8.251	3.673	2.259	0.249	0.460	0.310	0.208	0.238	0.312
5000	57.660	22.372	14.220	0.882	1.195	0.857	0.549	0.549	0.849
10000	267.487	88.759	63.031	1.903	3.722	2.057	1.527	1.234	1.797
20000	1112.684	349.326	221.882	3.620	4.647	4.047	2.414	2.528	4.698
50000	7403.691	2296.554	1398.853	10.352	12.544	10.737	6.532	6.527	15.132
100000	30015.740	8658.546	5556.871	23.982	26.712	23.331	14.202	13.981	47.361
200000	123737.732	37974.443	27468.237	60.893	61.220	53.814	32.292	32.189	178.559

Table 1: Execution time (ms) for sorting algorithms by array size.

5.2 Visual Analysis

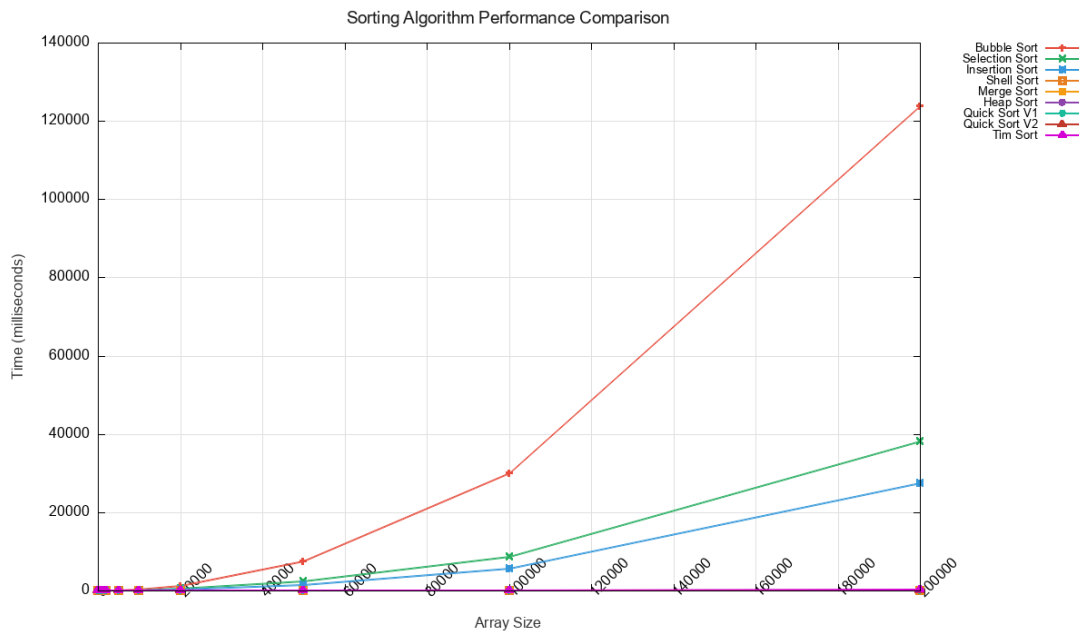


Figure 3: Overall performance comparison of all sorted algorithms.

The steep curve of Bubble Sort and other quadratic algorithms contrasts sharply with the flat lines of efficient algorithms, highlighting the importance of algorithmic choice.

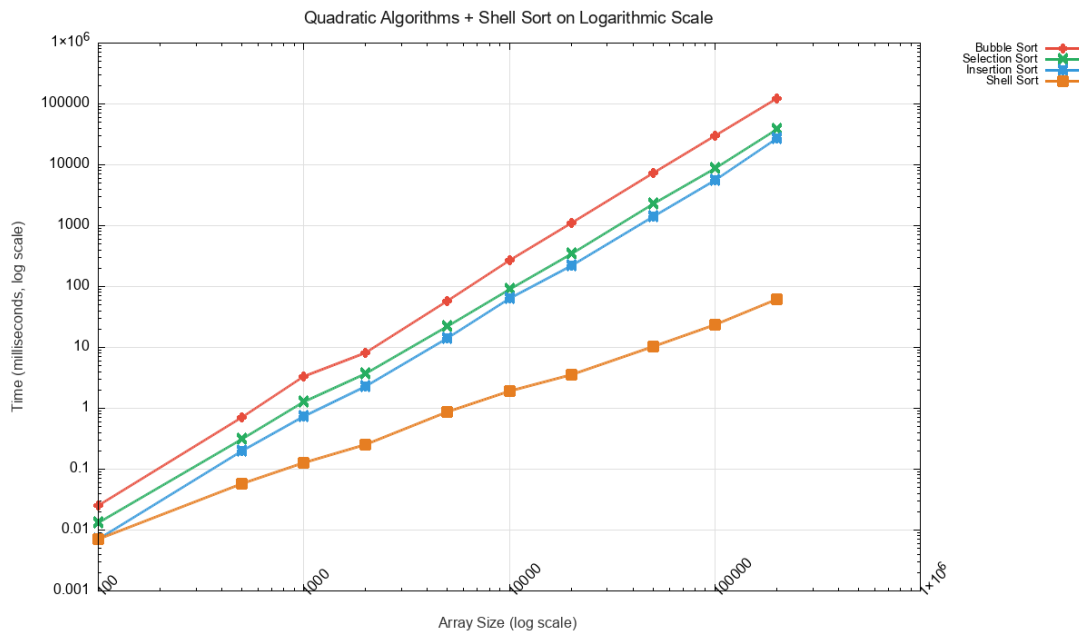


Figure 4: Logarithmic scale comparison.

The parallel lines for Merge, Heap, Quick, and Tim Sort confirm they belong to the same complexity class ($O(n \log n)$), while the steeper slopes of Bubble, Insertion, and Selection Sort indicate $O(n^2)$ complexity.

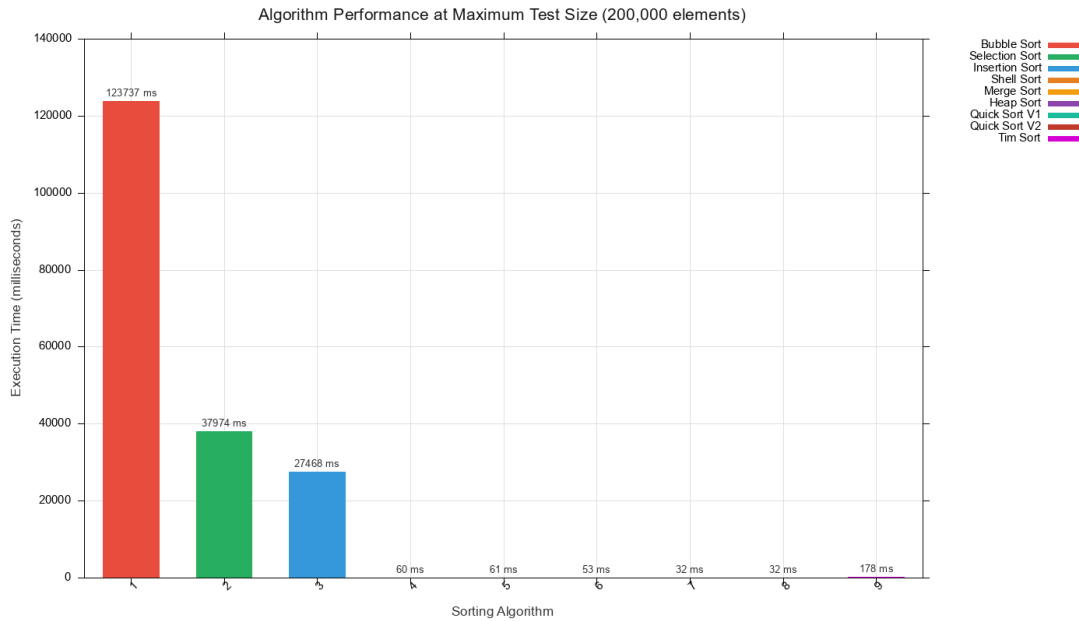


Figure 5: Bar chart comparison at maximum array size (200,000 elements).

This visualization emphasizes the massive performance gap, where quadratic algorithms take orders of magnitude longer than efficient ones.

5.3 Discussion

5.3.1 The $O(n^2)$ Wall

The data vividly illustrates the limitations of quadratic algorithms. At 200,000 elements, Bubble Sort takes over **2 minutes** (123.7s), while Quick Sort takes merely **32 milliseconds**. This is a performance difference of approximately **3800x**. Even Insertion Sort, the fastest of the quadratic group, is nearly 850x slower than Quick Sort at this scale. This confirms that quadratic algorithms are unsuitable for large datasets.

5.3.2 Quick Sort Optimization (V1 vs V2)

Both versions of Quick Sort perform similarly in terms of raw speed on random data (32.29ms vs 32.19ms at 200k). However, the value of V2 lies in its robustness. By eliminating tail recursion, V2 ensures that the stack usage remains $O(\log n)$. In V1, a worst-case pivot selection could lead to $O(n)$ stack frames, causing a crash on large inputs. V2 is therefore the "production-ready" implementation.

5.3.3 Tim Sort vs. The Rest

Tim Sort (178ms at 200k) is slower than pure Quick Sort (32ms) in this benchmark. This is expected for *random* data, as Tim Sort incurs overhead from identifying runs and merging. However, Tim Sort shines in real-world scenarios where data often contains partially sorted sequences. Its stability and worst-case $O(n \log n)$ guarantee make it a safer default for standard libraries than Quick Sort, which can degrade to $O(n^2)$ without careful pivot selection.

5.3.4 Key Insights at 100,000 Elements

At 100,000 elements, the performance gap is stark:

- **Quick Sort V2:** ≈ 14 ms
- **Merge Sort:** ≈ 27 ms
- **Insertion Sort:** ≈ 5557 ms (over 5 seconds)
- **Bubble Sort:** ≈ 30016 ms (over 30 seconds)

This demonstrates that for large datasets, $O(n^2)$ algorithms are practically unusable compared to their $O(n \log n)$ counterparts. The difference is not just theoretical; it translates to a user waiting milliseconds versus half a minute.

6 Conclusion

This study confirms the theoretical predictions of algorithm analysis with empirical data.

1. **Avoid $O(n^2)$ algorithms** for any dataset larger than a few thousand elements.
2. **Quick Sort V2** is the fastest general-purpose algorithm for random integer data in our tests, thanks to its low overhead and cache-friendly access patterns.
3. **Tail Recursion Optimization** is a critical technique for implementing robust recursive algorithms like Quick Sort.
4. **Tim Sort** offers a compelling balance of stability and performance, making it ideal for general-purpose library implementations where data characteristics are unknown.

For high-performance systems requiring raw speed on random data, **Quick Sort V2** is the recommended choice. For applications requiring stability and consistent performance across various data distributions, **Tim Sort** or **Merge Sort** are preferable.